

Steering the Ship

Encoding Your Team's
Standards as AI
Infrastructure



Who I Am: Patrick Liekhus

Role and Mission

Serve as the Principal AI Enablement Engineer at Provation, a global healthcare company improving clinician collaboration

The Core Risk of AI

Greatest danger in the AI coding era is not moving too slowly, but moving quickly in the wrong direction without guidance

Spec-Driven Development

Advocates for discipline by using clear specifications before writing code and steering files to protect architectural intent

Strategic Focus

Operates at the intersection of scalable architecture and practical AI application, for systems that are fast to deploy and solid

High-Stakes Experience

Deep expertise from highly regulated environments where AI must be carefully steered and precision is non-negotiable



Why Are Your Teams Producing Different Outputs From The Same Tool?

Same LLM, same prompt template, yet teams end up with vastly different code quality, structure, and testing discipline

Root cause: senior judgment lives in people's heads, not in systems—so juniors make different trade-off calls than seniors

Implicit standards create invisible inconsistency: each person has their own mental model of “good enough”

The gap widens with team size: as you scale, consistency degrades without explicit encoding of standards



What Happens When You Don't Steer?

Steering OFF: AI generates code with zero guardrails; assumes best practices; makes unconstrained trade-offs

Results: code that "works" locally but violates architecture, skips error handling, ignores security patterns, minimal test coverage (if any)

Steering ON: AI works within explicit constraints—required context, must-have standards, output format expectations

This comparison reveals the hidden cost of relying on model capability alone without human judgement infrastructure



What's Actually Different When Steering Is Active?



Structure: code follows predictable patterns and consistent organization—reviewers know what to expect

Completeness: error handling, logging, and edge-case tests are present by default, not afterthought

Compliance: security patterns, architecture boundaries, and regulatory constraints are automatically respected

Reviewability: faster PR cycles because code structure is predictable and intent is preserved in output

What's the Real Cost of Inconsistency?

Rework compounds: inconsistent code structure means higher review feedback, more iterations to production

Review drag: reviewers spend time understanding intent instead of focusing on logic; consistency enables faster decisions

Onboarding friction: new team members have no reference standards; they learn by imitation, perpetuating inconsistency

Compliance and audit risk: inconsistent patterns hide policy violations until expensive security review or audit surfaces them



Why Does Senior Judgment Stay Hidden?

Senior engineers have decades of implicit pattern recognition—they "just know" what's right, but can't always articulate it

This knowledge is fragile: it lives in individual heads, doesn't survive team turnover, can't be enforced without conversation

Onboarding seniors means spending months teaching juniors through code review, not through explicit standards

AI reveals this gap: models don't have implicit judgment, so they expose what you've never formalized



Steering Files Are Infrastructure, Not Documentation

Shift mindset: steering files aren't nice-to-have documentation or "coding guidelines" – they're official infrastructure for consistency

They encode the decision rights and quality expectations that let your team scale without degradation

Treat them like code: version-controlled, reviewed, tested against real tasks, refined from incident learnings

Neglecting them creates technical debt: shortcuts that seemed harmless compound into systemic inconsistency



Your Roadmap: Four Navigational Waypoints

Stop 1: Understand the anatomy of steering files—what goes in them and why

Stop 2: View code showing the before/after impact of steering on real code

Stop 3: Learn the adoption playbook—how to roll out steering with teams and measure success

Stop 4: Master governance: how to keep steering files fresh and evolving as your standards grow



What Actually Goes Into a Steering File?

Scope definition: state which decisions the AI should handle autonomously and which require human judgement

Role and accountability: tell the AI its specific responsibility(e.g., "you generate backend services, not full stack UI")

Context preconditions: specify what architecture, compliance, or domain context must be present before generating

Success criteria: define what "good" looks like in this domain so the AI can self-validate



Why being CRISP Matters?

C – Context: Set the stage by providing background information

R – Role: Define exactly who the AI is pretending to be. "Act as a senior software engineer."

I – Intent/Instruction: State the core task clearly. Explain what you need the AI to do, including any specific constraints or rules it must follow.

S – Style: Specify the tone, voice, or vibe of the output (e.g., professional, concise, humorous, or matching a specific brand voice).

P – Parameter/Product: Define the final output format, length, and structure (e.g., "Give me 5 bullet points," "Format this as a Markdown table," or "Keep it under 200 words").



What Context Preconditions Should You Require?

Architecture context: make AI aware of module boundaries, dependency direction, approved patterns before generating code

Domain context: ensure AI understands business rules, regulatory constraints, and non-functional requirements

Compliance context: surface security policies, data classification, and audit requirements upfront

History context: if past implementations exist, reference them so AI doesn't reinvent or deviate unnecessarily



What Are Your "Must" Standards?

Must standards are non-negotiable: security practices, architecture boundaries, test coverage thresholds

They come from compliance, regulatory, or critical business risk—violation is unacceptable

Examples: "all public APIs must have input validation", "database migrations must be reversible", "security credentials never hardcoded"

Make these explicit and checkable; they should be the first things code validation looks for



What Are Your "Should" Standards?

Should standards set defaults: preferred error handling patterns, logging conventions, performance considerations

They allow pragmatic trade-offs: if a deadline is tight, you can relax a "should" but never a "must"

Examples: "should include comprehensive error logging ", "should optimize for readability unless performance is critical"

These standards guide behavior while acknowledging context-dependent flexibility



What Are Your "Nice-to-Have" Standards?

Nice-to-haves improve quality when time permits: example logging, performance optimization, documentation

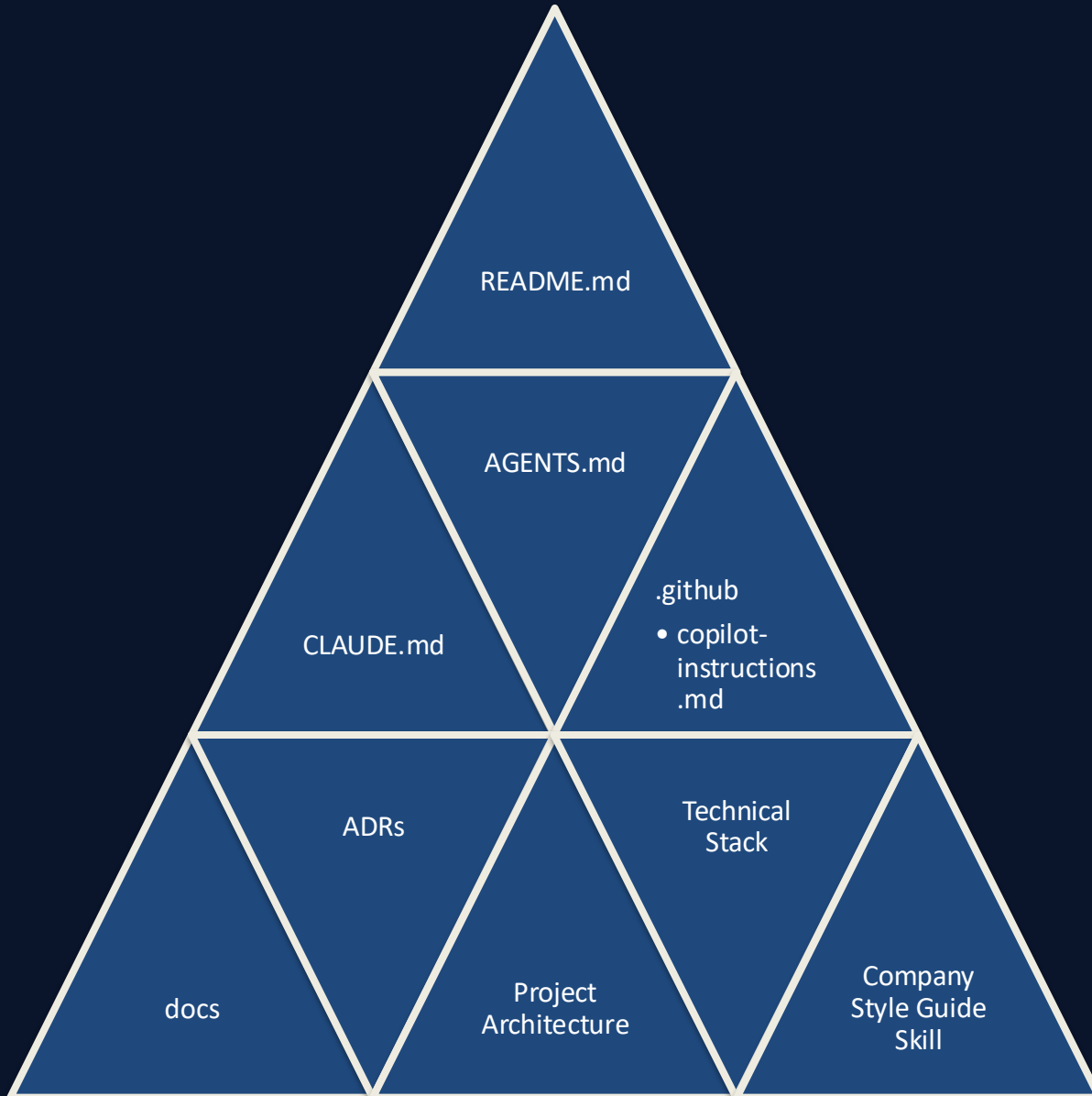
They act as quality boosters: if AI has capacity after meeting must/should standards, apply these

They make the difference between "acceptable" and "exemplary" output

Examples: "nice-to-have: includes performance monitoring instrumentation", "nice-to-have: comprehensive API documentation with examples"



How Does it Work?



WELCOME BACK

Sign in to continue building your best work.

Secure access, fast workflows, and everything you need in one place.

- End-to-end encrypted sessions
- Single-click project sync
- Live activity timeline

Sign In

Use your account credentials to access your dashboard.

Email address

name@company.com

Password

Enter your password

Show

☐ Remember me

[Forgot password?](#)

Log In

or

Continue with Google

Continue with GitHub

New here? [Create an account](#)

<> index.html

JS script.js

styles.css

create a login page

I'll scaffold a polished standalone login page in this workspace, then verify the files are in place so you can open it directly in a browser. I'm checking the folder contents first.

Reviewed demo-no-steering and searched for matching files

Next I'm creating three files: index.html, styles.css, and script.js with a responsive, production-style login UI and light client-side validation.

Created index.html, styles.css, and script.js

I've added the login page implementation; now I'll quickly verify the files exist and provide exactly how to run it.

Reviewed 2 files

Created a complete responsive login page in your workspace:

- index.html
- styles.css
- script.js

What's included:

- Modern two-panel layout with bold typography and atmospheric gradient shapes
- Email/password form with remember-me, forgot-password, and social login buttons

PROVATION WORKSPACE

Welcome Back

Sign in to continue your clinical documentation workflow with secure, fast access.

- SSO-ready sign-in experience
- Streamlined workflow access
- Protected clinical context

Sign In

Work Email

Password

☒ Keep me signed in

Sign In

[Need help signing in?](#)

The Angular scaffold is in place; now I'm wiring routes and replacing the default starter template with a focused login screen component.

Review Velocity: Why PR Cycles Get Faster

Review friction reduced: consistent structure means reviewer spots gaps quickly instead of reconstructing intent

Decision speed: because output is predictable and standards-aligned, approval decision takes less deliberation

Feedback loops cut: most common revision requests are pre-satisfied

Example: review goes from "this needs work" to "approved pending minor updates" on first review



Throughput: What Happens When Review Cycles Are Shorter?

First-pass acceptance rate increases: fewer requests for revision because standards are met upfront

Cycle time drops: code goes from AI to review to merge faster

Throughput improves: more completed work per sprint because less time spent on rework and iteration

Quality improves simultaneously: tighter standards mean less technical debt, fewer post release defects



Practical Tuning: Where Should You Tighten or Loosen Standards?

Start conservative: begin with firm standards, measure outcomes, relax only where data shows flexibility sage

Tighten on risk: if a domain has high compliance or security risk, keep standards tight

Loosen on repetition: if a pattern is proving easy and consistent, reduce steering detail

Measure and learn: track which steering rules actually prevent rework versus which are busywork



30-Day Rollout: From Pilot to Operational

Week 1: Baseline and planning—document current state standards, assemble pilot team, publish v1 steering file

Week 2: Pilot run—use steering file on real work, collect feedback, measure quality/review metrics

Week 3: Expand—roll out to second team or domain, refine based on pilot learnings, update steering file

Week 4: Harden—establish governance for steering updates, train teams, create escalation path for steering exceptions



Who Owns This? Roles and Accountability

Platform owner: maintains steering files, coordinates updates, ensures consistency across teams

Team leads: adapt steering for team context, coach on usage, provide feedback on effectiveness

Technical leadership: sets org-level standards that cascade to all steering files, resolves conflicts

Individual contributor: follows steering guidance, flags gaps or confusion, contributes learnings from code reviews



Treat Steering Updates Like Code

Version control: steering files live in repo with full history, allowing rollback if a change causes problems

Code review: substantive changes to steering files require review and approval before publishing

Changelog: document why steering changed so history is auditable

Examples: link steering changes to specific learning events—what incident triggered this refinement?



What Metrics Tell You Steering Is Working?



Rework rate: percentage of PRs requiring revision—steering should reduce this

Review cycle time: days from code ready to approved—steering should compress this

First-pass acceptance: percentage of PRs merged without revision—steering should increase this

Defect escape rate: bugs found in testing or production vs in review – steering should reduce this

What Steering Mistakes Kill Adoption?



Vague steering: so loose it doesn't constrain anything—wastes time without providing value

Rigid steering: so tight it prevents legitimate variation—creates frustration and workarounds

Stale steering: doesn't evolve as team learning accumulates—becomes irrelevant and ignored

Unowned steering: nobody is responsible for maintaining it—degrades as context changes

Give Teams the Tools to Succeed

Example steering files: share working templates teams can adapt for their domain

Checklists: provide review checklists for validating code meets steering standards

Awesome GitHub Copilot -
<https://github.com/github/awesome-copilot>

Skills.sh - <https://www.skills.sh>



This Is Infrastructure, Not Bureaucracy

Reframe: you're encoding senior judgment as reusable infrastructure, not adding red tape

The payoff: your entire team applies senior-level decision-making consistently, without a senior person reviewing every decision

Scale enabler: as you grow, steering lets new people contribute at your quality bar immediately, not after months of mentoring

Competitive advantage: teams with strong steering scale faster because they scale confidently



Your Three Cardinal Rules



Standardize intent first, then scale generation: be clear on the "what" and "why" before asking AI for "how"

Make standards explicit and testable: vague principles don't scale; concrete rules do

Evolve steering from incident learning: treat each escape defect as a steering improvement opportunity, not just a bug to fix

Thank You!

Questions?

Feedback?



Steering the Ship: Production-Ready AI Workflows with
Context, Rules, and Guardrails

